

1. Employee Payroll Management System

Objective

Develop a system to manage employee records and calculate monthly salaries for different types of employees such as full-time and part-time. The system should store all employee data permanently using file handling and handle invalid input safely through exception handling.

Key OOP Concepts

- **Encapsulation:** Private attributes such as `__emp_id` and `__salary` to protect data.
- **Abstraction:** Public methods like `calculate_pay()` and `update_salary()` provide a simple interface to work with employee data.
- **Inheritance:** Subclasses `FullTimeEmployee` and `PartTimeEmployee` inherit common functionality from the base class `Employee`.
- **Polymorphism:** The `calculate_pay()` method behaves differently for each subclass depending on the type of employee.

Features

1. Add new employees with automatic employee ID generation.
 2. Calculate monthly salary for full-time employees and hourly pay for part-time employees.
 3. Update salary and display employee details.
 4. Save and load all employee records from a file named `employees.txt`.
 5. Use `try/except` blocks to catch invalid input, negative salary, or file read/write errors.
-

2. Inventory Management System

Objective

Create a console-based system to manage product inventory for a store. It should allow adding, updating, deleting, and viewing products, with data saved permanently in a file.

Key OOP Concepts

- **Encapsulation:** Product details like `__product_id`, `__price`, and `__quantity` are private.
- **Abstraction:** Methods such as `add_product()` and `update_quantity()` hide the internal logic of how data is stored or updated.
- **Inheritance:** Specialized product types like `PerishableProduct` and `NonPerishableProduct` inherit from a base `Product` class.
- **Polymorphism:** The `apply_discount()` or `calculate_value()` method acts differently for each product type.

Features

1. Add new products with unique IDs and store them in a file named `inventory.txt`.

2. Search for products by ID or name.
 3. Update quantity or price of an existing product.
 4. Apply category-specific discounts using polymorphism.
 5. Handle exceptions for invalid input (negative price or quantity) and file errors.
-

3. Vehicle Rental Management System

Objective

Build a system to manage a fleet of vehicles (cars, bikes, trucks) available for rent. The program should allow users to rent or return vehicles and calculate rental charges.

Key OOP Concepts

- **Encapsulation:** Vehicle attributes such as `__vehicle_no`, `__rent_per_day`, and `__available` are private.
- **Abstraction:** Methods like `rent_vehicle()` and `return_vehicle()` hide details about calculations and availability changes.
- **Inheritance:** Derived classes `Car`, `Bike`, and `Truck` inherit from a base `Vehicle` class.
- **Polymorphism:** Each subclass overrides `calculate_rent()` to use its own pricing model.

Features

1. Add new vehicles and store them in a file named `vehicles.txt`.
2. Rent a vehicle by vehicle number and update its status.
3. Return a rented vehicle and calculate rent based on days.
4. Display all available or rented vehicles.
5. Use exception handling for invalid inputs (negative days, missing vehicles) and file operations.